

Arquitetura de Computadores

Eng. Informática, ESTiG/IPB

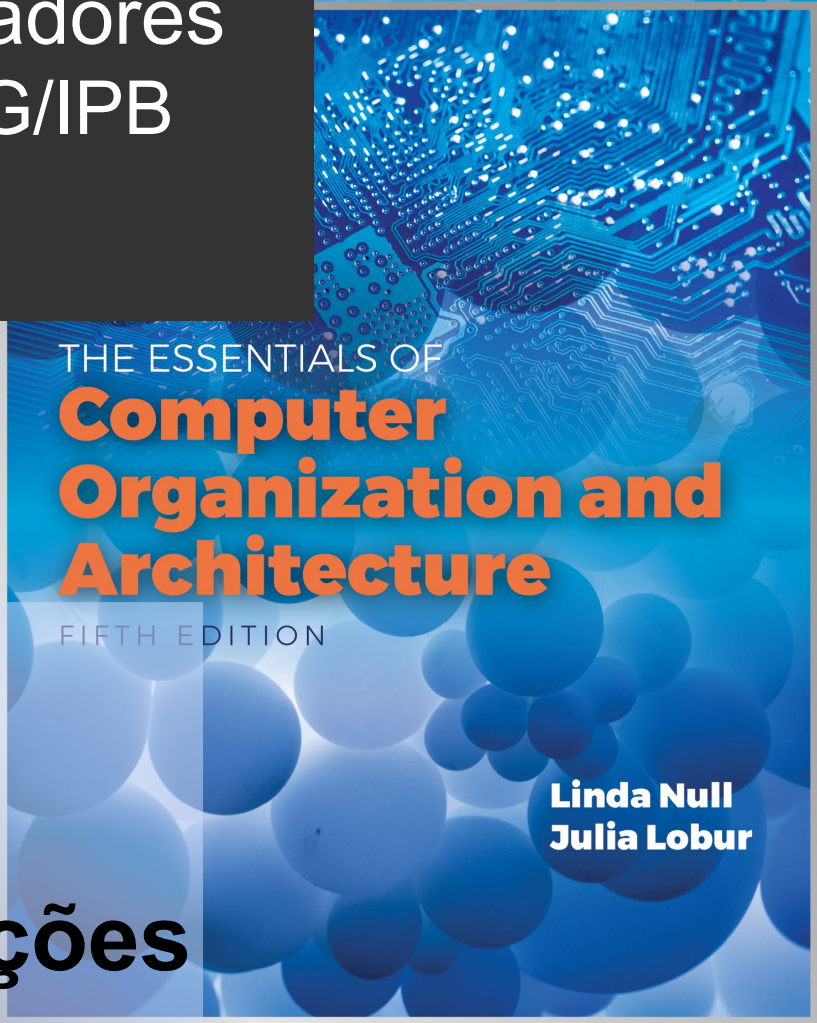
2024/2025

José Rufino

(baseado no Capítulo 5 do livro “The Essentials of Computer Organization and Architecture” de Linda Null e Julia Lobur)

Unidade 4:

Arquiteturas de Conjuntos de Instruções



THE ESSENTIALS OF
**Computer
Organization and
Architecture**

FIFTH EDITION

**Linda Null
Julia Lobur**

Objetivos



- Compreensão dos fatores envolvidos no desenho de *Arquiteturas de Conjuntos de Instruções (ISAs)*
- Familiarização com vários *Modos de Endereçamento de Memória* e conceitos subjacentes
- Compreensão do conceito de *Encadeamento de Instruções* e o seu efeito no *Desempenho*
- Conhecer as propriedades que distinguem as arquiteturas RISC das arquiteturas CISC

4.1 Introdução



- Tomando como base os conceitos previamente apresentados para a arquitetura MARIE ...
 - ... faremos uma análise mais aprofundada dos diferentes *formatos de instrução*, dos *tipos de operandos* e dos *métodos de acesso à memória*
 - ... bem como será analisada a relação entre a *organização do sistema* e os *formatos de instrução*
- **Resultado:** aquisição de um conhecimento mais sólido do que é a arquitetura de um computador

4.2 Formatos de Instrução

- Decisões de Desenho



- os **conjuntos de instruções** distinguem-se por:
 - nº de bits de cada instrução (**MARIE: $16 = 4 + 12$**)
 - forma de armazenamento dos operandos na CPU: estrutura baseada em *stack* ou **registos** (**MARIE**)
 - nº de operandos explícitos por instrução (**MARIE: 1**)

4.2 Formatos de Instrução

- Decisões de Desenho



- os **conjuntos de instruções** distinguem-se por:
 - localização dos operandos (**reg-to-reg**, **reg-to-mem**, **mem-to-reg**, mem-to-mem; **MARIE ? (input, output)**)
 - tipos de operações (incluindo a definição das que acedem à RAM; **MARIE: inst X**)
 - tipo e tamanho dos operandos (operandos podem ser **endereços**, **números**, **caracteres**; **MARIE ? (IO)**)

4.2 Formatos de Instrução

- Decisões de Desenho



- as **ISAs** são avaliadas / comparáveis segundo:
 - memória principal ocupada pelos programas
 - complexidade das instruções
 - trabalho de decodificação por instrução
 - complexidade das tarefas a executar
 - tamanho das instruções (em bits)
 - número total de diferentes instruções

4.2 Formatos de Instrução

- Decisões de Desenho



- fatores no **desenho de um conjunto de instruções**:
 - tamanho da instrução (curto/longo, fixo/variável)
 - curto: menos RAM, *fetch* mais rápido, menos instruções, operandos mais pequenos, menos operandos por instrução
 - fixo: decodificação mais fácil, desperdício de espaço
 - tamanho de instrução vs número de operandos
 - tamanho de inst. fixo não implica número de oper. fixo ...
 - modos de endereçamento suportados
 - direto (**MARIE**), indireto (**MARIE**), indexado, ...

4.2 Formatos de Instrução

- Decisões de Desenho



- fatores no **desenho de um conjunto de instruções**:
 - organização da memória
 - endereçável ou byte vs endereçável à palavra
 - memórias com palavras > 1 byte (16/32/64 bits) suportam frequentemente endereçamento ao byte em vez de à palavra, permitindo instruções que manipulam caracteres individuais
 - número, organização e papel dos registos
 - registos genéricos vs dedicados (FPU), janelas de registos
 - ordem de endereç./armazen. dos bytes numa palavra
 - *endianess: big endian vs little endian* (ver a seguir →)

4.2 Formatos de Instrução

- Ordem dos Bytes (*Endianness*)

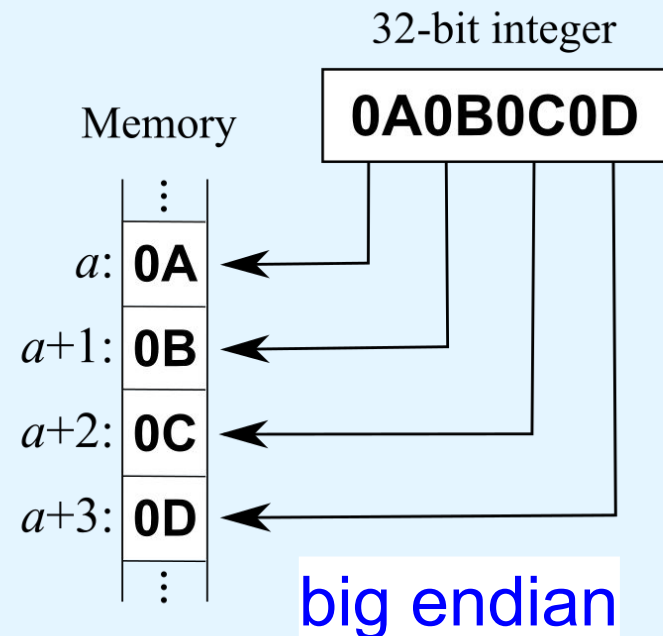
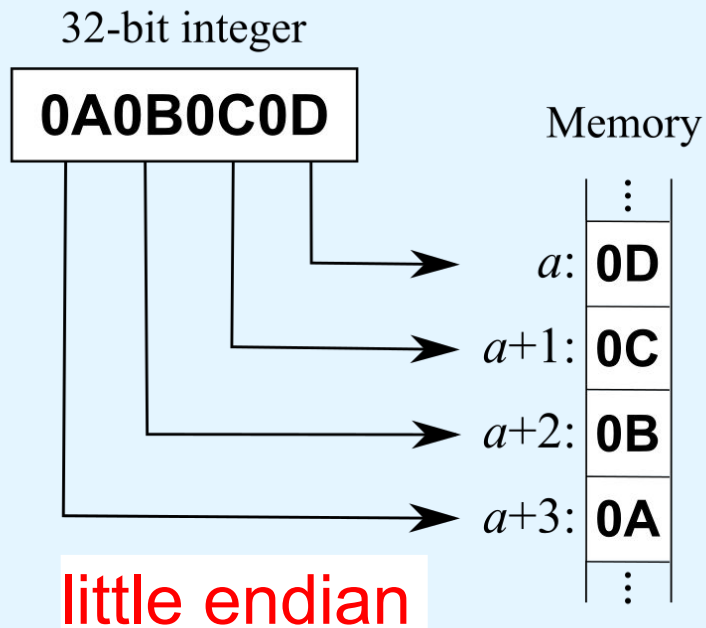


- na maioria das arquiteturas a memória é endereçada ao byte => necessário um *standard* para a ordem de armazenamento/endereçamento de conjuntos de bytes
 - como é armazenado um inteiro com mais de 1 byte ?
ou como é enviado através de uma rede de dados ?
- abordagem **little endian**: o byte **menos** significativo é guardado/enviado primeiro, seguido dos mais significativos
 - exemplo: CPUs Intel, ARM; protocolo de rede SMB
- abordagem **big endian**: o byte **mais** significativo é guardado/enviado primeiro, seguido dos menos significativos
 - exemplo: CPUs Motorola, MIPS, PowerPC; redes TCP/IP

4.2 Formatos de Instrução

- Ordem dos Bytes (*Endianness*)

- exemplo: disposição de um inteiro de 32 bits (4 bytes)



[<https://en.wikipedia.org/wiki/Endianness>]

4.2 Formatos de Instrução

- Ordem dos Bytes (*Endianess*)

checkpoint: exercícios
LEBE1 a LEBE7

- *big endian*:
 - mais natural (ex: inserção de dígitos no multibanco)
 - sinal (-/+) determinável pela inspeção do primeiro byte
 - *strings* e inteiros são armazenados da mesma forma
 - manipulação eficiente de bitmaps (imagens, arquivos)
- *little endian*:
 - arqs. c/ aritmética de + precisão, + rápida e + fácil/ implementação
 - conversão de 32 para 16 bits por mera truncagem ...
 - exemplo: número 0xMMNNmmnn; BE guarda MM NN mm nn; LE guarda nn mm NN MM; com LE, bastar ficar com os 2 primeiros bytes para fazer a conversão de 32 bits para 16 bits
 - leituras de tamanho errado (e.g., via *cast*) podem produzir valores corretos
 - exemplo: número 0x00005678; BE guarda 00 00 56 78; LE guarda 78 56 00 00; com LE, ler por engano apenas os primeiros 2 bytes ainda preserva o valor certo !

4.2 Formatos de Instrução

- Armazenamento Interno na CPU



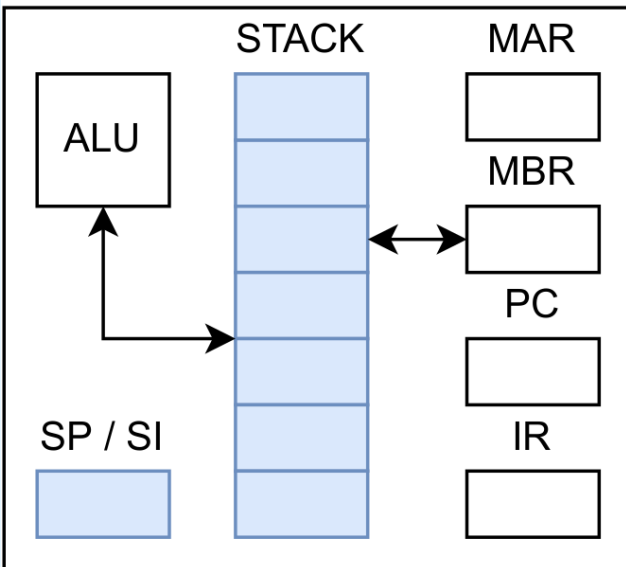
- outra decisão a tomar no desenho de um arquitetura é a forma de armazenamento de operandos na CPU
- a este respeito, há 3 abordagens principais:
 1. arquitetura baseada em **stack**
 2. arquitetura baseada em **um registo acumulador**
 3. arquitetura baseada em **registos genéricos**
- as mesmas representam diferentes compromissos:
 - i) simplicidade/custo do desenho do hardware
 - ii) velocidade de execução dos programas
 - iii) conveniência de utilização (programação)

4.2 Formatos de Instrução

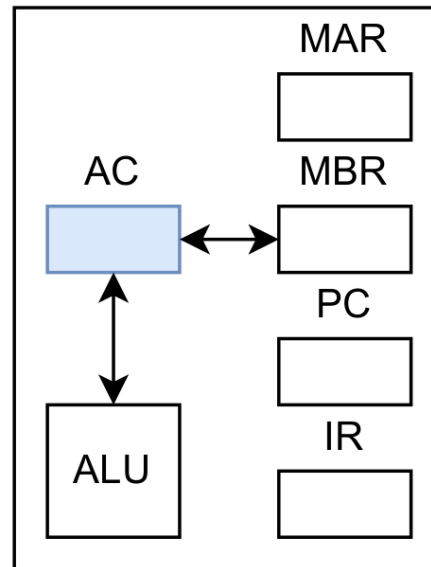
- Armazenamento Interno na CPU

- abordagens ao armazenamento interno na CPU:

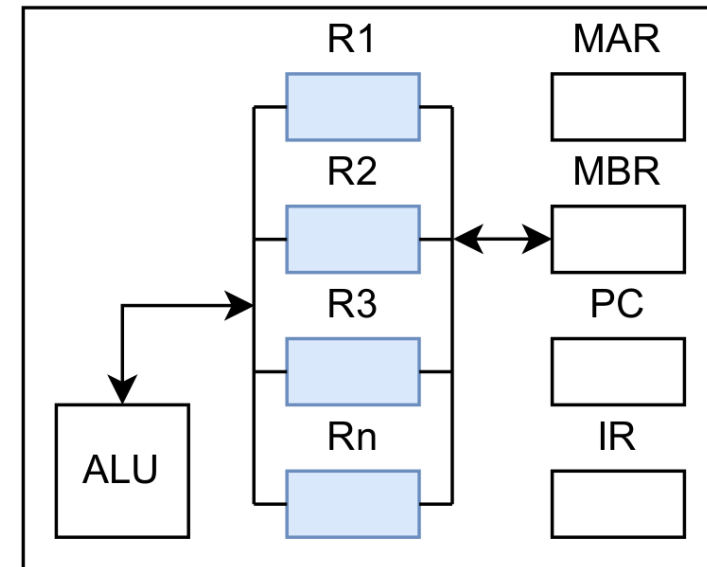
Stack-based



Accumulator-based



General Purpose Register-based



4.2 Formatos de Instrução

- Armazenamento Interno na CPU



- arquitetura de **Stack** (exemplo: MSP)
 - operandos: implícitos na *stack* da CPU, explícitos na RAM
 - suporta modelos simples de avaliação de expressões (*postfix*)
 - a *stack* não pode ser acedida aleatoriamente => código < eficiente
 - um registo SI/SP (Stack Index/Pointer) indica o topo atual da stack (a próxima célula livre, ou a última célula ocupada)
- arquitetura de **Acumulador** (exemplo: MARIE)
 - registo acumulador usado como um operando implícito
 - numa operação binária, o outro operando reside na RAM
 - um só registo acumulador => elevado tráfego de acesso à RAM

4.2 Formatos de Instrução

- Armazenamento Interno na CPU



- arquitetura de **Registos Genéricos (GPR)**
 - tem um (ou mais) conjunto(s) de vários registos genéricos
 - mantém o maior número possível de operandos em registos
=> melhor desempenho que na arquitetura de Acumulador ...
(< contenção no barramento de acesso à memória principal)
 - permite aos compiladores geração de código mais eficiente
 - resulta em instruções mais longas (necessidade de identificação dos registos) => maiores tempos de *fetch* e descodificação
 - atualmente, a generalidade dos sistemas reais é de tipo GPR

4.2 Formatos de Instrução

- Armazenamento Interno na CPU



- **tipos de sistemas GPR**, por localização de operandos:
 - **memória-memória**: dois ou três operandos em memória
 - DEC VAX **Add X,Y** // $X = X + Y$
 - **registo-memória**: um operando em registo e outro em memória
 - Intel, Motorola (CISC ...), MARIE
 - Load R,X; **Add R,Y**; Store R,X // $X = X + Y$
 - **load-store**: nenhum operando em memória (sempre em registos)
 - SPARC, MIPS, ALPHA, PowerPC (RISC ...)
 - Load R1,X; Load R2,Y; **Add R1,R2**; Store R1,X // $X = X + Y$
- O número de operandos e de registos disponíveis tem um impacto direto no tamanho das instruções (ver a seguir)

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- **instruções de dimensão fixa** (exemplo: MARIE):
 - decodificação rápida (simplicidade), execução rápida (*pipelining*), desperdício de espaço (bits não usados)
- **instruções de dimensão variável**:
 - decodificação lenta (complexidade), economia de espaço
- na prática:
 - coexistência, na mesma arquitetura, de 2 ou 3 dimensões de instrução, assentes em padrões de bits de fácil decodificação
 - dimensão da instrução e da palavra iguais => alinhamento perfeito na memória => acesso + rápido, < desperdício RAM
 - instruções de dimensão variável => desperdício inevitável, pois as instruções estão sempre alinhadas à palavra

exemplo: instruções de 24 bits em palavras de 32 bits

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- **formatos de instrução comuns:**
 - *opcode* (zero endereços)
 - *opcode* + 1 endereço (tipicamente de RAM; **MARIE**)
 - *opcode* + 2 endereços
 - 2 registos
 - 1 registo + 1 endereço de RAM
 - *opcode* + 3 endereços
 - 3 registos
 - combinações de registos e endereços de RAM

4.2 Formatos de Instrução

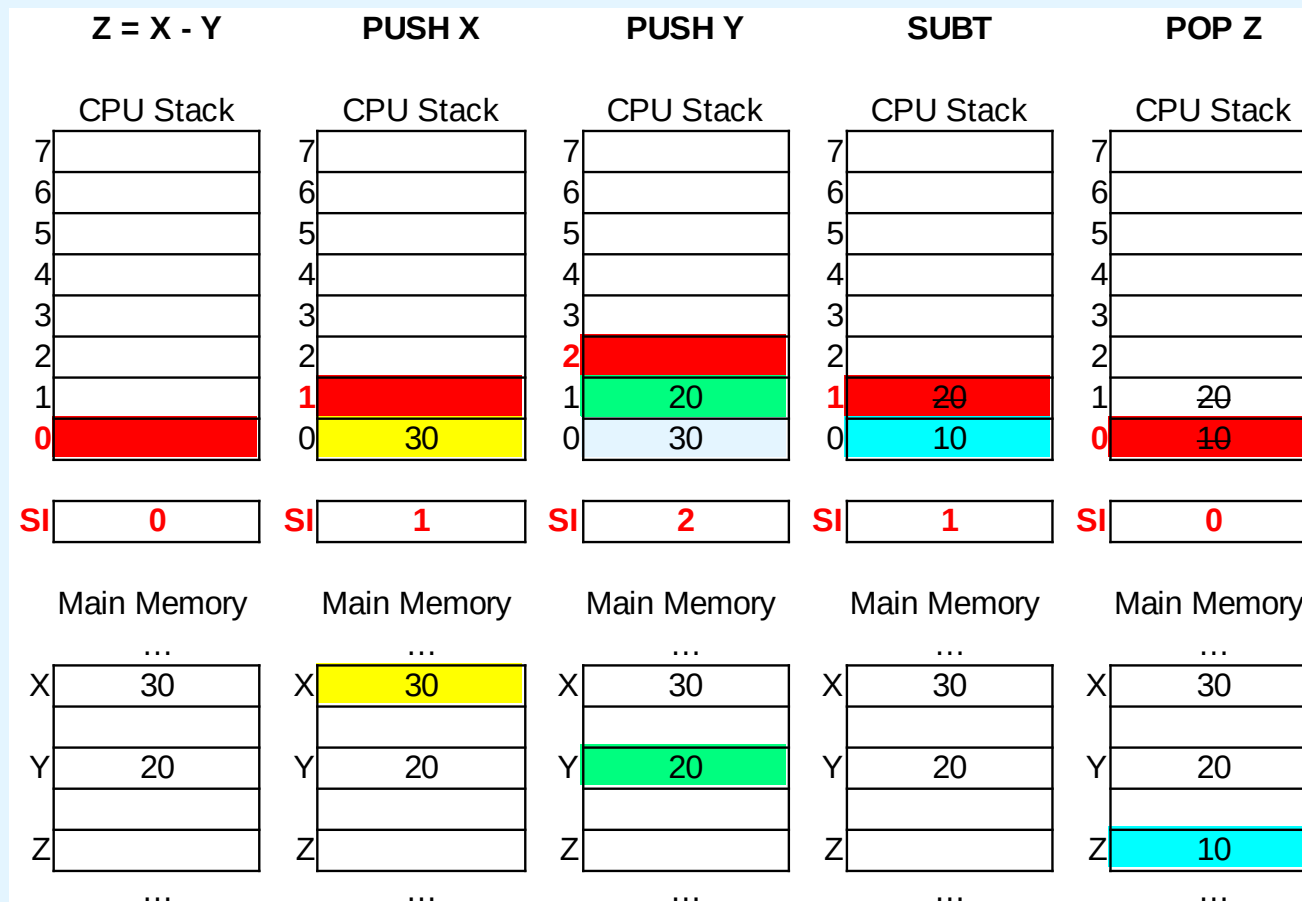
- Número de Operandos e Dimensão das Instruções

- **arquiteturas baseadas em *stack*:**
 - instruções com um ou zero operandos explícitos
 - acesso à memória: “**PUSH X**” e “**POP X**”
 - um operando explícito (**X**) é um endereço da mem. principal, um operando implícito é o topo da *stack* ($STACK[SI]$)
 - **PUSH X** : copia $M[X]$ para o topo da *stack* e faz a *stack* crescer ($STACK[SI] = M[X]; SI++$); **POP X** : copia para $M[X]$ o topo da *stack* e a *stack* decresce ($M[X] = STACK[SI-1]; SI--$)
 - instruções aritméticas binárias: “**ADD**”, “**SUBT**”, ...
 - operandos e resultado implícitos no topo da *stack*
 - **SUBT** : “retira” os dois valores do topo da *stack* e insere a diferença ($STACK[SI-2] = STACK[SI-2] - STACK[SI-1]; SI--$)

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- arquiteturas baseadas em *stack* (cont.):
 - exemplo de avaliação de uma expressão ($Z = X - Y$)



4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- **arquiteturas baseadas em *stack* (cont.)**
 - estas arquiteturas obrigam a pensar de forma diferente
 - vulgarmente, as expressões aritméticas são escritas em notação ***infix***, como por exemplo $Z = X - Y$
 - mas para serem avaliadas numa arquitetura de stack, têm de ser reescritas em notação ***postfix***: $Z = XY-$
 - dispensa uso de parêntesis e a tradução para assembly de uma arquitetura de stack é trivial (ver slide a seguir)
 - algumas expressões ***infix*** e respetivas expressões ***postfix***

$(X + Y) \times (W - U) + 2$

$X Y + W U - \times 2 +$

$Z = (X \times Y) + (W \times U)$

$Z = X Y \times W U \times +$

checkpoint: exercícios SBA1 a SBA3

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- exemplos de cálculo de uma mesma expressão com diferente número de operandos explícitos (considerando apenas **operações aritméticas**):
 - numa ISA baseada em stack, a expressão *postfix*

Z = X Y × W U × +

pode ser avaliada com o código

```
PUSH X
PUSH Y
MULT
PUSH W
PUSH U
MULT
ADD
POP Z
```

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- exemplos de cálculo de uma mesma expressão com diferente número de operandos explícitos (cont.):

- numa ISA de 1-endereço (MARIE), a expressão *infix*

$$Z = X \times Y + W \times U$$

pode ser avaliada com o código

```
LOAD X
MULT Y
STORE TEMP
LOAD W
MULT U
ADD TEMP
STORE Z
```

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- exemplos de cálculo de uma mesma expressão com diferente número de operandos explícitos (cont.):

- numa ISA de 2-endereços (Intel), a expressão *infix*

$$Z = X \times Y + W \times U$$

pode ser avaliada com o código

```
LOAD R1,X
MULT R1,Y
LOAD R2,W
MULT R2,U
ADD R1,R2
STORE Z,R1
```

NOTA: numa ISA de 2-endereços, normalmente, um dos operandos é um registo, como neste exemplo (R1,R2)

4.2 Formatos de Instrução

- Número de Operandos e Dimensão das Instruções

- exemplos de cálculo de uma mesma expressão com diferente número de operandos explícitos (cont.):
 - numa ISA de 3-endereços (mainframes), a expressão
infix **Z = X × Y + W × U**
pode ser avaliada pelo código

```
MULT R1,X,Y  
MULT R2,W,U  
ADD  Z,R1,R2
```

checkpoint: exercícios
NOIS1 a NOIS5

Tendo menos instruções, será que este programa executaria mais rápido que a versão da ISA baseada em *stack* ?

4.2 Formatos de Instrução

- Códigos de Operação Expansíveis

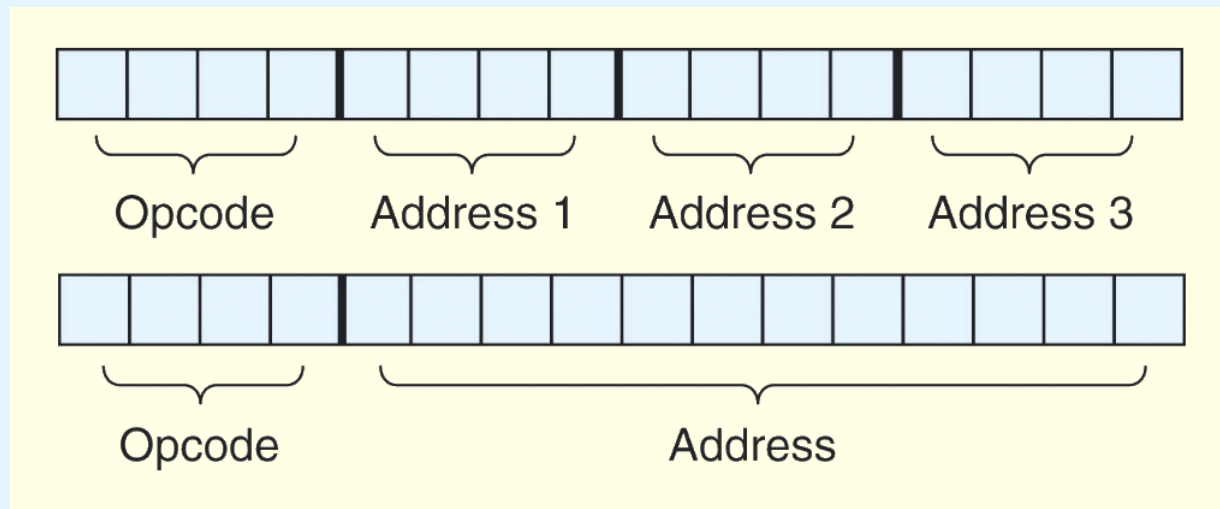


- o tamanho de uma instrução depende do número de operandos suportados pela arquitetura
- em qualquer conjunto de instruções nem todas as instruções exigem o mesmo número de operandos
- operações que não exigem operandos, tal como **HALT**, levam a algum desperdício de espaço, se for usado um tamanho fixo para as instruções
- uma forma de minimizar este problema é usar *opcodes* (códigos de operação) *expansíveis*

4.2 Formatos de Instrução

- Códigos de Operação Expansíveis

- seja um sistema com 16 registros e RAM de 4K palavras
 - são necessários 4 bits para identificar um registro
 - são necessários 12 bits para cada endereço da RAM
- com instruções de 16 bits de comprimento, e **opcode de tamanho fixo**, alguns formatos de instrução possíveis são:



- até 16 instruções com 3 registros como operandos
- até 16 instr. c/ 1 endereço de memória como operando

4.2 Formatos de Instrução

- Códigos de Operação Expansíveis

- se o **tamanho do opcode** puder variar, então podem ser criadas mais de 16 instruções:

0000	R1	R2	R3	}	15 3-address inst.
....					
1110	R1	R2	R3	}	14 2-address inst.
1111	0000	R1	R2		
				}	31 1-address inst.
1111	1101	R1	R2		
1111	1110	0000	R1	}	16 0-address inst.
					
1111	1110	1111	R1	}	
1111	1111	0000	R1		
				}	
1111	1111	1110	R1		
1111	1111	1111	0000	}	
					
1111	1111	1111	1111	}	

4.2 Formatos de Instrução

- Códigos de Operação Expansíveis

checkpoint:
exercícios
EO1 to EO4

- o esforço acrescido na decodificação das instruções pode ser minimizado com um algoritmo eficiente, como:

```
if (leftmost four bits != 1111 ) {  
    Execute appropriate three-address instruction  
}  
else if (leftmost seven bits != 1111 111 ) {  
    Execute appropriate two-address instruction  
}  
else if (leftmost twelve bits != 1111 1111 1111 ) {  
    Execute appropriate one-address instruction  
}  
else {  
    Execute appropriate zero-address instruction  
}
```


4.3 Tipos de Instruções



As instruções de uma ISA são de várias categorias:

- movimentação de dados (entre registos e/ou memória)
- aritmética (inteira e de vírgula flutuante)
- lógica booleana (AND, OR, XOR, NOT)
- manipulação de bits (aritméticas, lógicas, *shift*, rotação)
- entrada/saída (*programmed IO*, *interrupt-driven IO*, DMA)
- transferência de controlo (saltos, *skips*, inv. de funções)
- fins específicos (proc. de strings, proteção, gest. cache)

Qual a categoria de cada instrução MARIE?

4.4 Endereçamento

- Modos de Endereçamento



- **modos de endereçamento (ME):**
 - formas de indicar (a localização de) um operando
 - podem especificar
 - uma constante
 - um registo
 - uma posição de memória
 - **endereço efetivo**: verdadeira localização do operando
 - alguns modos de endereçamento permitem determinar *dinamicamente* o endereço efetivo de um operando

4.4 Endereçamento

- Modos de Endereçamento



- endereçamento **Imediato**:
 - os dados (operandos) fazem parte da própria instrução
 - ótimo desempenho, mas total inflexibilidade
 - NOT 008 ==> 008 é o valor a negar
- endereçamento **Direto** (MARIE):
 - *endereço efetivo* (de memória) dos dados indicado na instrução
 - bom desempenho, com alguma flexibilidade
 - NOT 008 ==> 008 é o endereço (de memória) do valor a negar
- endereçamento **Direto por Registro**:
 - semelhante ao endereçamento Direto, mas com registros
 - dados num registro, cujo identificador é embebido na instrução
 - NOT 008 ==> 008 identifica o registro com o valor a negar

4.4 Endereçamento

- Modos de Endereçamento



- endereçamento **Indireto** (MARIE):
 - a instrução contém um **endereço** de uma posição de memória
 - nessa posição está guardado o **endereço efetivo** dos dados
 - o **endereço** na instrução é a localização de um **apontador**
 - NOT 008 ==> negar o valor cujo endereço está em 008
- endereçamento **Indireto por Registro**:
 - semelhante ao endereçamento Indireto, mas com registros
 - um registro guarda o **endereço efetivo** dos dados
 - NOT 008 ==> negar o valor cujo endereço está no registro 008

4.4 Endereçamento

- Modos de Endereçamento



- endereçamento **Indexado**:
 - a instrução contém um **endereço** de uma posição de memória
 - um registo (implícito ou explícito) guarda um **deslocamento**
 - **endereço efetivo** dos dados = **endereço** + **deslocamento**
 - NOT 008 ==> negar o valor cujo endereço é 008 + R1
- endereçamento **com Base**:
 - utiliza uma lógica dual à do endereçamento indexado
 - registo base com endereço, instrução com deslocamento
- endereçamento **com Stack**:
 - assume que os dados estão no topo de uma *stack*

4.4 Endereçamento

- Modos de Endereçamento

- há muitas outras variantes, não referidas aqui ...
- exemplo: **LOAD 800**, com R1 como registo de índice

checkpoint:
exercícios AM1 a AM3

Memory	
800	900
...	
900	1000
...	
1000	500
...	
1100	600
...	
1600	700

R1 800

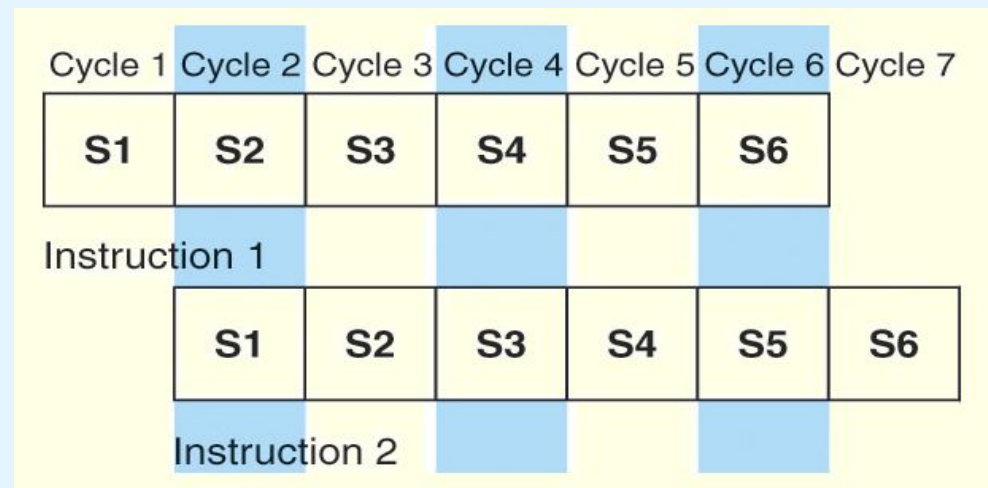
Mode	Value Loaded into AC
Immediate	800
Direct	900
Indirect	1000
Indexed	700

4.5 Encadeamento de Instruções (ILP)

- algumas CPUs dividem o ciclo *fetch-decode-execute* em vários passos ou *estágios* de pequena duração
- estes estágios podem, muitas vezes, ser executados em paralelo, com o intuito de aumentar o *throughput*
 - *throughput* : nº de instruções concluídas por u.t.
- esta execução em paralelo é denominada de encadeamento de instruções ou *instruction-level pipelining* (ILP)
- ILP é similar a uma linha de produção por estágios ...
 - diferentes estágios decorrem em simultâneo
 - cada estágio alimenta(-se) o (do) estágio seguinte (anterior)

4.5 Encadeamento de Instruções (ILP)

- exemplo:
 - ciclo **fetch-decode-execute** dividido nos seguintes passos
 1. ler a instrução
 2. decodificar o *opcode*
 3. calcular os endereços efetivos dos operandos
 4. ler os operandos
 5. executar a instrução
 6. guardar o resultado
 - existe um *pipeline* com $k=6$ estágios S_k , um por passo
 - por cada ciclo de relógio é concluído um dos passos, e os $k=6$ estágios sobrepõem-se
==>



4.5 Encadeamento de Instruções (ILP)

- o ganho de desempenho produzido pelo encadeamento de instruções (ILP) é determinado da seguinte forma:
 - considere-se um *pipeline* com k estágios, de duração igual
 - cada estágio consome t_s unidades de tempo (ciclos de relógio)
 - considerem-se n instruções, que vão percorrer o *pipeline*
 - a primeira instrução exige um tempo $t_i = k \cdot t_s$ para concluir
 - as restantes $n - 1$ instruções saem do *pipeline* uma por ciclo
 - o tempo total para conclusão das $n - 1$ instruções será $(n - 1) \cdot t_s$
 - tempo para completar n instruções em *pipeline* com k estágios:
 $(k \cdot t_s) + (n - 1) \cdot t_s = (k + n - 1) \cdot t_s$ [em u.t. ou ciclos de relógio]

4.5 Encadeamento de Instruções (ILP)

- o ganho de desempenho produzido pelo encadeamento de instruções (ILP) é determinado da seguinte forma (cont.):
 - tempo necessário para executar n instruções sem *pipeline*:
$$n \cdot t_i = n \cdot (k \cdot t_s)$$
 - ganho de desempenho (aceleração ou *speedup*) para n instruções: tempo sem *pipeline*, dividido pelo tempo com *pipeline*
$$S_n = n \cdot t_i / (k + n - 1) \cdot t_s = n \cdot (k \cdot t_s) / (k + n - 1) \cdot t_s$$
$$= n \cdot k / (k + n - 1)$$
 - quando n tende para infinito, $(k + n - 1)$ aproxima-se de n , obtendo-se então o máximo ganho teórico, que é k :

$$MAX(S_n) = S_{n \rightarrow \infty} = n \cdot k / (k + n - 1) = k$$

checkpoint: exercícios ILP1 a ILP3

4.5 Encadeamento de Instruções (ILP)

- as equações anteriores assumem certos **pressupostos**
 - 1) a duração de cada estágio do *pipeline* é similar
 - 2) a arquitetura é capaz de ler instruções e dados (operandos) em paralelo ==> barramentos separados
 - 3) o *pipeline* é mantido preenchido durante o período total de execução de um programa; mas nem sempre é possível:
 - quando é incerto o resultado de um teste a uma condição (devem entrar na pipeline as instruções da alternativa V ou F ?)
 - quando há conflito de recursos entre instruções no pipeline
 - quando uma instrução na pipeline depende do resultado de outra que ainda está na pipeline (ainda não concluiu)

4.5 Encadeamento de Instruções (ILP)

- o encadeamento de instruções pode ser suspenso (*stalled*) ou interrompido (com anulação de passos):
 - saltos condicionais

Time Period →	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction: 1	S1	S2	S3	S4									
2		S1	S2	S3	S4								
(branch) 3			S1	S2	S3	S4							
4				S1	S2	S3							
5					S1	S2							
6						S1							
8							S1	S2	S3	S4			
9								S1	S2	S3	S4		
10									S1	S2	S3	S4	

- “soluções”: i) previsão de saltos (CPU), ii) *fetch* das instruções correspondentes às duas alternativas lógicas (CPU), iii) adiamento dos saltos (reordenação/inserção de instruções por compiladores)

4.5 Encadeamento de Instruções (ILP)

- razões para a interrupção do encadeamento (cont.):
 - conflitos de recursos
 - exemplo: uso do bus em simultâneo por LOAD e STORE
 - soluções: i) a instrução mais atrasada é obrigada a aguardar pelo recurso comum; ii) duplicação do n° de unidades do recurso
 - dependências de dados
 - uma instrução depende do resultado ainda não disponível de outra
 - soluções: i) hardware que detecta a situação e gera instruções NOOP para alimentar o *pipeline*, ii) re-ordenação das instr. p/ compiladores
- estas causas não podem ser completamente eliminadas

4.6 ILP vs Outras Técnicas de Aceleração

- **Instruction Level Parallelism (ILP):**
 - execução simultânea de várias instruções no mesmo núcleo de CPU; implementado (**automat/**) no hardware (CPU), recorrendo a técnicas como i) *(super-)pipelining* e ii) *super-escalaridade*
- ***super-pipelining*:** quando a CPU tem mais que uma pipeline
- ***super-escalaridade*:** a CPU tem várias unidades funcionais do mesmo tipo (e.g. ALU); normal/ combinada com super-pipelining
- ***Explicit Parallel Instruction Computers (EPIC)*:** várias instrs. pequenas que podem ser executadas em paralelo, empacotadas numa macro-instrução na compilação (instr. 128 bits Intel Itanium)
- **Program Level Parallelism (PLP):** o programador particiona (**explicita/**) o programa em secções para executarem em paralelo; necessita de um sistema paralelo (CPU multi-core ou cluster)

4.7 Arquiteturas RISC vs CISC



- **RISC (Reduced Instruction Set Computer)**
 - i) poucas instruções distintas, com acesso explícito à memória por instruções específicas (*load* e *store*)
 - ii) instruções com a mesma dimensão (n° bits)
 - iii) n° de ciclos de relógio para decodificação e execução é semelhante para as várias instruções
- **CISC (Complex Instruction Set Computer)**
 - i) variados tipos de instruções, umas com acesso explícito e outras com acesso implícito à memória
 - ii) dimensão das instruções variável
 - iii) tempo de *fetch-decode-execute* variável

4.7 Arquiteturas RISC vs CISC



- a diferença entre sistemas CISC e RISC evidencia-se na equação de desempenho de um computador:

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

- os sistemas **RISC** reduzem o tempo de execução através da **redução do nº de ciclos por instrução**
- os sistemas **CISC** melhoram o desempenho através da **redução do número de instruções por programa**
- ambos beneficiam de maior frequência de CPU

4.7 Arquiteturas RISC vs CISC

(*) ver a
secção 3.10

- a simplicidade do conjunto de instruções das máquinas **RISC** possibilita o uso de **unidades de controlo do tipo hard-wired** (> velocidade)
 - instruções diretamente implementadas em hardware (*)
- o conjunto de instruções mais complexo e variável das máquinas **CISC** obriga à utilização de **unidades de controlo (*) microprogramadas** (> flexibilidade)
 - instruções implementadas com base em micro-instruções (*)
- com **instruções de tamanho fixo**, os sistemas **RISC** adequam-se melhor ao **encadeamento de instruções** (*pipelining*) e à **execução especulativa**

4.7 Arquiteturas RISC vs CISC



- considerem-se os seguintes fragmentos de código:

CISC

```
mov ax, 10  
mov bx, 5  
mul bx, ax
```

RISC

```
mov ax, 0  
mov bx, 10  
mov cx, 5  
Begin add ax, bx  
loop Begin ; loops cx times
```

- o nº total de ciclos para a versão CISC é:
 $(2 \text{ movs} \times 1 \text{ ciclo}) + (1 \text{ mul} \times 30 \text{ ciclos}) = 32 \text{ ciclos}$
- enquanto na versão RISC será:
 $(3 \text{ movs} \times 1 \text{ ciclo}) + (5 \text{ adds} \times 1 \text{ ciclo}) + (5 \text{ loops} \times 1 \text{ ciclo}) = 13 \text{ ciclos}$
- precisando de menos ciclos, e sendo cada ciclo mais curto, RISC oferece velocidades de execução superiores

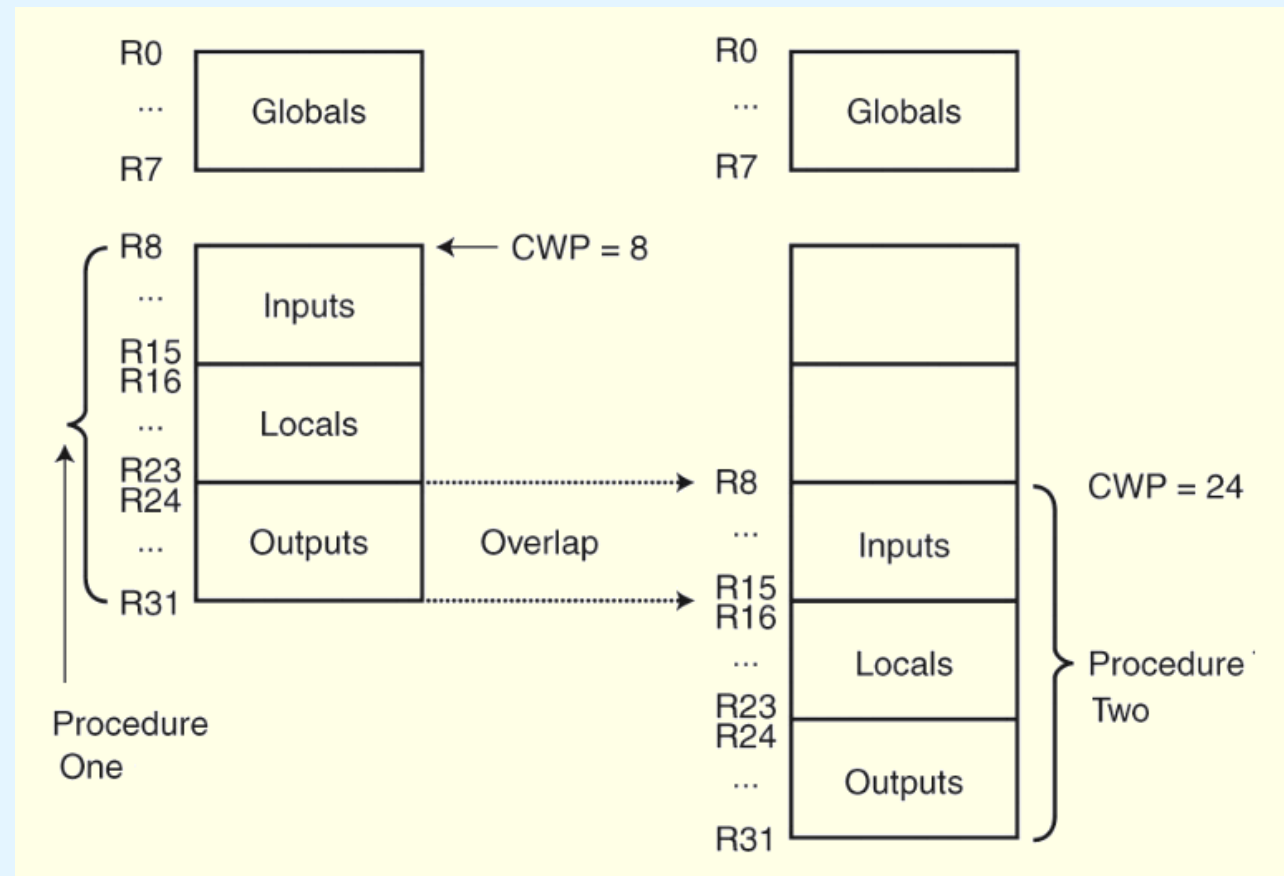
4.7 Arquiteturas RISC vs CISC



- por serem arquiteturas *load-store*, as arquiteturas RISC requerem um grande nº de registos na CPU
- esses registos
 - permitem acesso rápido aos dados durante a execução sequencial dos programas
 - podem ser usados para reduzir a sobrecarga da passagem de parâmetros para subrotinas
 - em vez de se passarem os parâmetros na RAM (como no MARIE, ou usando uma stack na RAM)
 - ... utiliza-se um subconjunto dos registos da CPU (internos à CPU, logo de acesso muito + rápido) →

4.7 Arquiteturas RISC vs CISC

- os registos podem ser sobrepostos quando são invocadas várias rotinas
- um apontador (CWP) indica qual a **janela de registos** que está ativa



CWP = current window pointer

valores típicos em CPUs RISC:
16 janelas, de 32 registos cada

4.7 Arquiteturas RISC vs CISC

- RISC vs CISC (resumo)

Hoje em dia, muitos CPUs apresentam, em simultâneo, características RISC e CISC

RISC	CISC
Multiple register sets, often consisting of more than 256 registers	Single register set, typically 6 to 16 registers total
Three register operands allowed per instruction (e.g., <code>add R1, R2, R3</code>)	One or two register operands allowed per instruction (e.g., <code>add R1, R2</code>)
Parameter passing through efficient on-chip register windows	Parameter passing through inefficient off-chip memory
Single-cycle instructions (except for <code>load</code> and <code>store</code>)	Multiple-cycle instructions
Hardwired control	Microprogrammed control
Highly pipelined	Less pipelined
Simple instructions that are few in number	Many complex instructions
Fixed length instructions	Variable length instructions
Complexity in compiler	Complexity in microcode
Only <code>load</code> and <code>store</code> instructions can access memory	Many instructions can access memory
Few addressing modes	Many addressing modes

4.8 Exemplos de ISAs

- Intel (Arquitetura Real de Tipo GPR+CISC)

- formato de instrução: dimensão variável, 2 endereços, arquitetura *register-memory* (um oper. em RAM, outro em registo)
- suporte ao encadeamento de instruções (*pipelining*):
 - o 1º Pentium tinha dois *pipelines* com $k=5$ estágios;
 - cada novo Pentium tinha um *pipeline* maior que o seu antecessor, tendo o Pentium IV chegado aos 24 estágios
 - o Itanium (IA-64) tem um *pipeline* com 10 estágios
- suporte a diferentes modos de endereçamento:
 - 8086: 17 formas de endereçar a memória, muitas das quais correspondem a variantes das apresentadas neste capítulo
 - Pentium *: novos modos, além de compat/ com os anteriores
 - Itanium, de filosofia RISC, suporta apenas endereçamento indireto por registo com incremento posterior opcional

4.8 Exemplos de ISAs

- MIPS (Arquitetura Real de Tipo GPR+RISC)

- MIPS = *Microprocessor Without Interlocked Pipeline Stages*
- formato de instrução: dimensão fixa, 3 endereços, arquitetura *load-store* (acesso à RAM apenas por LOAD e STORE; todas as outras instruções usam registos como operandos)
- suporte ao encadeamento de instruções (*pipelining*):
 - {R2000, R3000} / {R4000, R4400}: *pipelines* de $k=5$ / 8 estágios
 - R1000: *pipelines* com n° variável de estágios; $k=5$ estágios p/ instruções com inteiros, $k=7$ estágios p/ instruções de vírgula flutuante e $k=6$ estágios para instruções LOAD e STORE
- suporte a diferentes modos de endereçamento:
 - a ISA suporta endereçamento com base; o *assembler* suporta imediato, por registo, direto, indireto por registo e indexado

4.8 Exemplos de ISAs

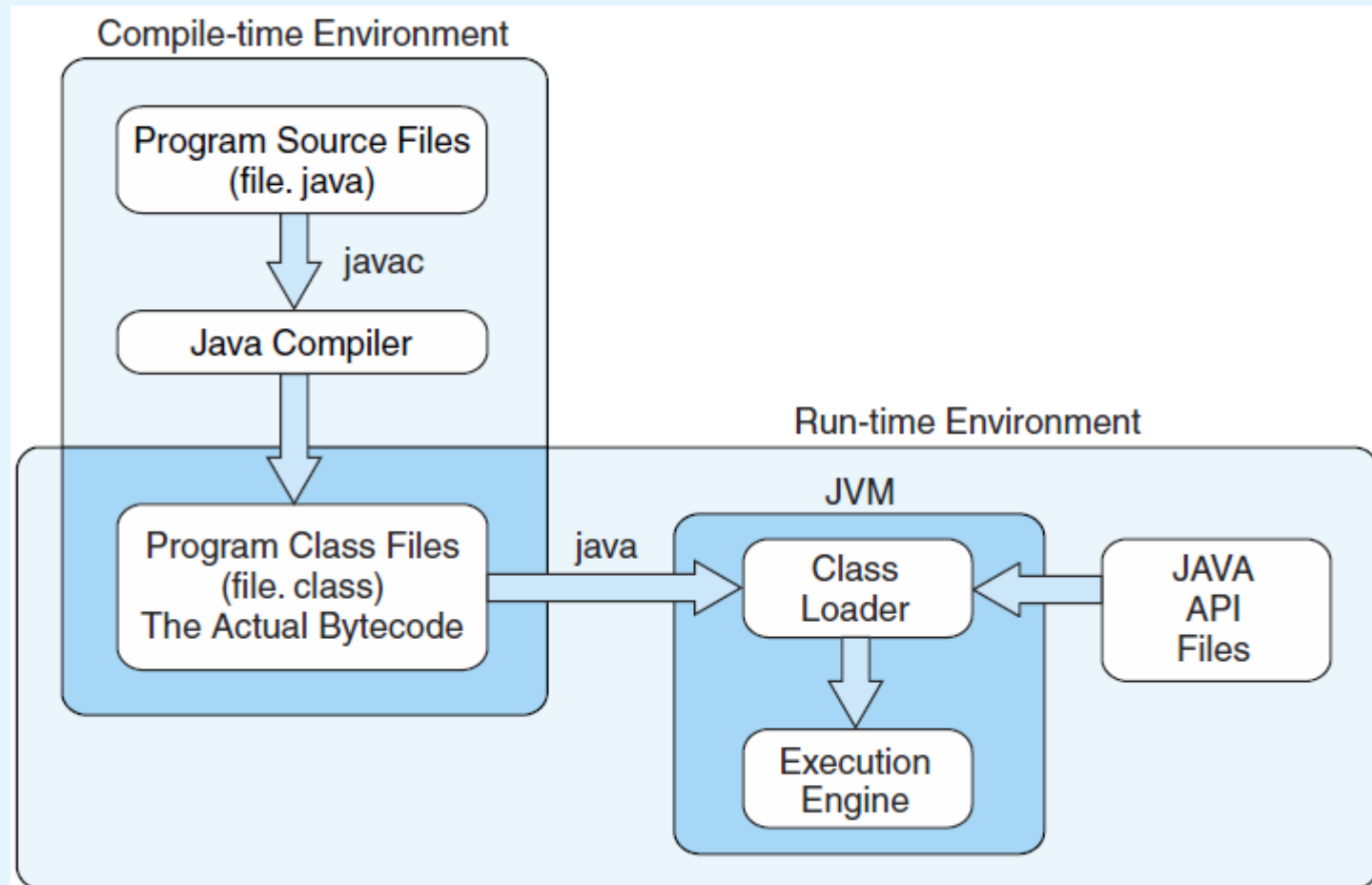
- Java (Arquitetura Virtual de Tipo Stack)



- o Java permite escrever programas que correm numa máquina em software - *Java Virtual Machine* (JVM)
- uma JVM é escrita numa linguagem nativa para uma vasta gama de processadores, incluindo MIPS e Intel
- tal como uma máquina real, a JVM tem uma *ISA* própria
- esta *ISA* foi desenhada por forma a ser compatível com a arquitetura de qualquer máquina onde a JVM corre
- programas em Java são compilados para *bytecode*
- uma JVM interpreta o bytecode e, para cada instrução, invoca uma rotina que a implementa em código nativo

4.8 Exemplos de ISAs

- Java (Arquitetura Virtual de Tipo Stack)



4.8 Exemplos de ISAs

- Java (Arquitetura Virtual de Tipo Stack)



- o *bytecode* Java é uma linguagem baseada em *stack*
- muitas instruções contemplam zero operandos
- a JVM tem 4 registros que permitem o acesso a 5 regiões de uma memória principal virtualizada
 - todas as referências à memória são deslocamentos a partir do endereço contido nos registros; não se permite apontadores nem referências absolutas
- o Java foi desenhado para garantir interoperabilidade e não desempenho (consequência do uso da *stack*)

Conclusões



- as ISAs são distinguidas pelo nº de bits e pelo nº de operandos por instrução, bem como pela localização, tipo e tamanho dos operandos
- a ordem dos bytes (*endianness*) é uma questão arquitetural importante, com impactos diversos
- a CPU pode armazenar dados com base em:
 1. uma *stack*
 2. um registo acumulador
 3. múltiplos registos de uso genérico

Conclusões

- as instruções podem ter tamanho fixo ou variável
- para enriquecer o conjunto de instruções de uma arquitetura de instruções de tamanho fixo, podem usar-se códigos de operação expansíveis
- os modos de endereçamento podem incluir:
 - imediato
 - registro
 - indireto
 - baseado
 - direto
 - indireto por registro
 - indexado
 - *stack*

Conclusões



- um *pipeline* com k estágios pode, em teoria, produzir um ganho de desempenho k
- conflitos de recursos e saltos condicionais impedem na prática que tal desempenho seja alcançado
- as principais diferenças dos sistemas RISC relativamente aos CISC incluem o tamanho fixo e reduzido das instruções; as arquiteturas RISC são arquiteturas load-store; por isso os sistemas RISC incluem níveis elevados de pipelining
- as arquiteturas Intel, MIPS e JVM são bons exemplos dos conceitos apresentados neste capítulo